BT4012 Fraudulent Bitcoin Transaction Detection

Wang TianAi | Code Link: https://github.com/WangTianai920/BT4012-Kaggle

1. Introduction

Background: Bitcoin transactions can be used to move funds for illicit purposes such as money laundering. Bitcoin's pseudonymous addresses make attribution difficult, while its transaction graph offers relational evidence (Weber et al., 2019). As such, this competition requires us to detect fraud Bitcoin transactions to help investigators prioritise suspicious payment flows.

Data Overview: Given data form a transaction graph: 203,769 transactions (nodes) and 234,355 directed payment-flow edges, each transaction with 165 anonymised features (local attributes plus one-hop neighbourhood aggregates) and a time step. Training set covers time steps 1–35, with only 31,235 labelled (27,591 licit, 3,644 illicit; 11.7% illicit) and 110,537 unlabelled. The test set holds the 15,329 labelled transactions of steps 36–49.

Overarching Approach: I began by comparing conventional classifiers, tree ensembles and neural networks using temporal validation, then developed the strongest approaches through graph-based feature engineering, recency weighting and controlled hyperparameter experiments. Among these, two key strategies that stood out were to use unlabelled transactions as a separate observation status and to combine a strong tree model with a bounded graph-neural correction. To reduce overfitting and prediction variability, I also used regularisation and tested multiple random seeds. Finally, I selected two submissions that balanced public ROC-AUC with differences in target formulation, graph use and training weights, aiming to reduce reliance on a single modelling strategy.

2. Methodology

2.1 Data Preprocessing and Feature Engineering

During the preprocessing stage, I found no duplicate transaction IDs, train–test ID overlap, duplicate edges, self-loops, missing attribute cells or infinite values. Consequently, the supplied transaction attributes required no imputation.

To handle the 78% unlabelled rows, I treated them separately for the two submitted predictions:

- The binary model, 0929-1, excluded unlabelled rows from supervised fitting.
- The hybrid model, 1019-6, retained all training rows and distinguished three observation statuses: licit(0), illicit(1) and unlabelled(2). In particular, each fraud score evaluates the illicit probability *conditional on being labelled*:

$$s = P(illicit)/P(illicit) + P(licit)$$

To measure how much a feature's typical value changes across time periods, for each feature I computed:

$$\text{Drift score} = \frac{\max(\text{period medians}) - \min(\text{period medians})}{\text{standard deviation across transactions in steps 1–14}}.$$

Based on further experiments, I decided to remove ten features with the largest normalised shifts in period medians during steps 1–14, aiming to reduce reliance on unstable features:

Feature	(Normalised) drift score
feat_101	3.2157
feat_103	3.116
feat_100	2.4782
feat_136	2.3087
feat_135	2.2381
feat_134	2.2368
feat_149	1.845
feat_139	1.8104
feat_113	1.6937

Table 1. Removed features' drift scores

After such, the remaining 155 attributes were combined with 17 structural features describing degrees, unique predecessors and successors, source/sink status, directed two-hop paths and neighbour-degree statistics. These features capture how transactions receive and distribute funds, providing context unavailable from individual attributes alone.

Additionally, 1019-6 added nine higher-order topology features, consisting of five chain-position features, two node-removal fragmentation features and two biconnected-block features. These describe positions in payment chains, connectivity bottlenecks and locally interconnected structures.

2.2 Model Development and Validation

Both final models used histogram gradient boosting, but differed in their targets, graph components and temporal weighting.

0929-1 — binary trees + historical neighbour-risk ensemble + period alignment

Firstly, I used a binary tree model to estimate each transaction's risk. It uses HistGradientBoostingClassifier, an ensemble of decision trees, where successive trees improve the predictions of the existing ensemble.

After the first model scores transactions, I aggregated those scores through graph connections, followed by using specialist tree models to learn whether these neighbourhood patterns help predict illicit activity. During the specialist training, a model fitted through period 28 generates predictions for periods 29–35 and the specialists learn from that later block. This time separation prevents the first-stage model from using those rows' labels when generating their risk features. Moreover, predictions were combined conservatively by blending the base model with original and enhanced neighbour-risk specialists:

```
simple = 0.9 * base + 0.1 * original_specialist
anchor = 0.975 * simple + 0.025 * enhanced_specialist
```

bagged = 0.975 * anchor + 0.025 * bootstrap_average
(bootstrap_average averages specialists fitted to historical resamples stratified by period and label)

For period alignment, I converted the blended probability to log-odds z, then applied:

$$z_{\text{adjusted}} = z - 0.10(m_t - m_{\text{reference}})$$

Here, m_t is that forecast period's median log-odds; $m_{reference}$ is the median of the forecast-period medians, giving each period equal weight. This helps preserve rankings within each period, while adjusting comparisons between periods.

1019-6 — three-status tree teacher + bounded graph residual

Firstly, I used a histogram gradient-boosted tree model as a "teacher" to estimate each transaction's probability of being licit, illicit or unlabelled. The teacher uses 600 boosting iterations, a learning rate of 0.03 and a maximum of 15 leaves per tree.

To account for temporal changes, I gave recent transactions greater training weight. The base weight halves for every 20 periods before period 35. Transactions before period 18 receive an additional reduction, halving their weight for every three periods before that point. This retains older observations while reducing their influence. I then rescaled the unlabelled status to contribute 40% of total training weight, preventing its large number of observations from dominating training.

After training the teacher, I used a directed graph neural network to refine its predictions. The network aggregates information from incoming and outgoing neighbours separately, then uses learned gates to control their contributions. Rather than producing an entirely independent prediction, I added a bounded correction to the teacher's class log-probabilities:

$$z_c = \log(p_{\text{teacher},c}) + \tanh(r_c)$$

where r_c is the network's correction for class c. The tanh function limits each correction to between −1 and +1. The network's output head is initialised to zero, so its predictions initially match the teacher for transactions with supplied attributes. The teacher remains fixed while the network trains for 60 epochs using weighted three-status cross-entropy.

The adjusted class scores are converted into probabilities using SoftMax. I then obtain the binary illicit-risk score by excluding the unlabelled probability and renormalising:

$$s = P(illicit)/P(illicit) + P(licit)$$

Finally, I trained the graph component with five random seeds(42, 101, 202, 303 and 404) on top of the same teacher and combined their predictions in log-odds space. This reduces sensitivity to random initialisation and training variation.

$$s_{\text{final}} = \text{sigmoid}\left(\frac{1}{5}\sum_{k=1}^{5}\text{logit}(s_k)\right).$$

Validation and tuning

Random splits may leak future patterns, so all model fitting used expanding-window temporal validation. For origin o, the model trains on steps ≤ o and evaluate on the next 7–14 steps (origins 14, 17, 21, 24, 28, plus confirmation origins 17, 18, 20). ROC-AUC measures ranking across thresholds, so it was the primary metric under imbalance; I also inspected within-period and later-horizon AUC. Feature pruning, scalers and weights use history rows only. Table 2 shows the graph residual's contribution on these windows.

Window (train ≤ o → evaluate)	Labelled / illicit	Teacher	Teacher + graph residual
o = 14 → 15–28	10,311 / 1,537	0.9869	0.9881
o = 17 → 18–24	6,751 / 840	0.9806	0.9815
o = 21 → 22–35	11,705 / 1,773	0.9655	0.9691
o = 24 → 25–31	4,009 / 841	0.9968	0.9967
o = 28 → 29–35	6,028 / 1,102	0.9976	0.9975
Mean		0.9855	0.9866

Table 2. ROC-AUC on temporal validation windows

2.3 Model Exploration and Final Strategy

Exploration process review:

1. *Model-family screening.* In the initial two-fold temporal comparison, HGB achieved a mean ROC-AUC of 0.97463, compared with 0.96909 for XGBoost, 0.94393 for Extra Trees and 0.90639 for logistic regression. These results motivated my further development of focusing on HGB-based models.
2. *Feature engineering and temporal weighting.* I refined the HGB approach through recency weighting, two-hop graph features and history-based drift pruning. Historical neighbour-risk specialists and period-level score alignment were subsequently incorporated, producing 0929-1, which achieved a public ROC-AUC of 0.94997.
3. *Training-target reformulation.* I extended binary classification to distinguish licit, illicit and unlabelled transactions, allowing unlabelled observations to contribute to training as a separate status. The resulting submission, 0929-3, achieved a public ROC-AUC of 0.95223, supporting further exploration of this formulation.
4. *Integration of graph learning.* I supplemented the 181-input tree teacher with a bounded neural graph correction to capture additional neighbourhood information. Public ROC-AUC increased from 0.95389 for the teacher alone (1008-4) to 0.95578 for the hybrid model (1008-5), suggesting that graph learning could complement the engineered features.

5. *Training-window refinement.* Finally, I compared full-history training with shorter historical windows to investigate sensitivity to temporal change. These experiments led to 1019-6, which progressively downweighted observations before period 18. This soft weighting strategy achieved the highest recorded public ROC-AUC of 0.95967.

Fine-tuning methods:

- *Hyperparameter search*. Temporal-window experiments varied tree size, learning rate, iterations, feature sampling, recency half-life and unknown-status weight. Some boosting and pseudo-labelling searches used inner temporal splits before outer evaluation. Final recipes use fixed iteration counts rather than test-driven early stopping.
- *Controlled comparisons*. Many follow-up trials changed one component against a fixed incumbent; later candidates combined promising components. Fixed blend weights and shallow trees limited flexibility.
- *Validation gates*. Gates considered mean horizon AUC and tolerated only limited slice degradation; rules evolved during development. For example, submission 0929-1's alignment improved mean 14-step AUC from 0.950210 to 0.950328, while its worst slice lost 0.000412, within the preset 0.001 tolerance. More aggressive alignment settings failed this guardrail.
- *Seed control*. Changing the GNN seed of the 1008-5 recipe reduced public AUC by 0.00109. Later experiments used multiple seeds, and the final hybrid averages five seeds to reduce initialization sensitivity.

Model selection strategy: I selected 1019-6 for its strongest public score and 0929-1 for its different target formulation and graph strategy, and relatively lower prediction correlation with 1019-6, attempting to balance predictive strength with model diversity.

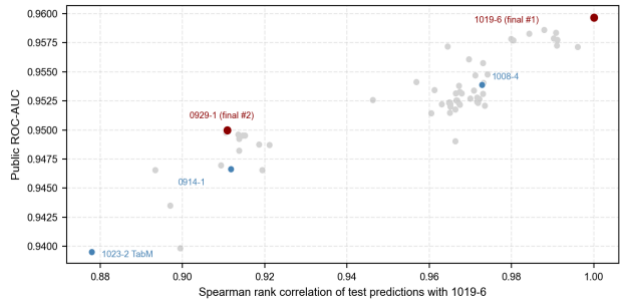


Figure 1. Public AUC against test-prediction rank correlation

Candidate	Model family	Public AUC	Pearson / Spearman Corr.
1019-6	three-status HGB + graph residual, soft window	0.95967	1.000 / 1.000
1008-4	three-status HGB only	0.95389	0.993 / 0.973
0929-1	binary HGB + neighbour-risk ensemble + alignment	0.94997	0.982 / 0.911
0914-1	recency-weighted XGBoost	0.94663	0.980 / 0.912
1023-2	standalone TabM	0.93953	0.956 / 0.878

Table 3. Final-pair shortlist comparison

3. Results and Discussion

3.1 Leaderboard Performance

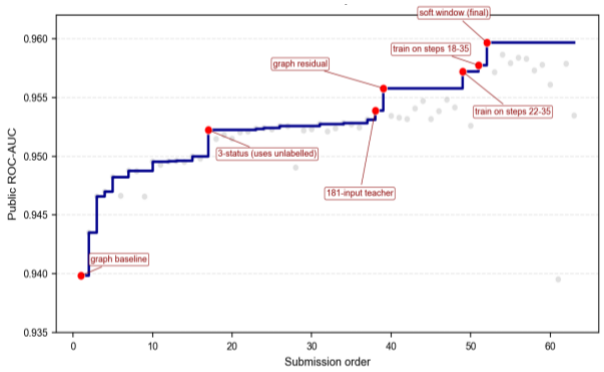


Figure 2. Public ROC-AUC over all submissions

Submission	Change	Public AUC
0912-1	HGB + structural graph features	0.93985
0912-2	+ recency weighting (half-life 20)	0.94350
0929-3	3-status model using unlabelled rows	0.95223
1008-4	181-input teacher with chain / cut / block features	0.95389
1008-5	+ bounded directed graph residual	0.95578
1019-5	train on steps 18–35 only	0.95772
1019-6	soft window: full weight 18–35, 3-step fade before	0.95967
1020-2	1019-6 with 10-step base recency half-life	0.95862

Table 4. Essential Milestones with key changes

It turns out that the three largest gains came from changing the problem formulation:

1. the unlabelled third class (+0.0023 over the previous best);
2. the graph residual (+0.0019);
3. restricting training to recent history (+0.003 for comparable 5-seed ensembles; +0.004 for 1019-6 over 1008-5).

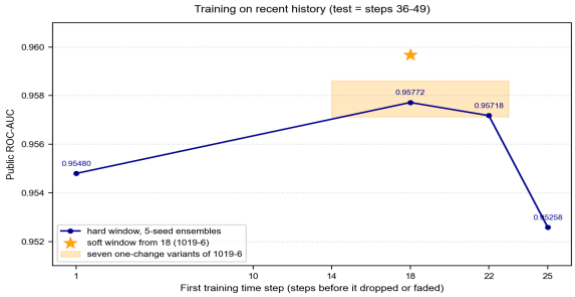


Figure 3. Public ROC-AUC against the first training time step.

Figure 3 illustrates the training-window effect. Starting training at period 18 improved AUC from approximately 0.9548 to 0.9577, whereas starting at period 25 reduced it to 0.9526. These results motivated further development around recent-history weighting rather than increasingly aggressive removal of older data.

Therefore, I explored a soft-window strategy that retained older observations but progressively reduced their training weights. This produced 1019-6, which achieved the highest recorded public AUC of 0.95967 and became my primary final submission.

3.2 Historical Validation Performance

I also compared the models on historical transactions with known labels. For periods 22–35, both models trained using data through period 21. For periods 29–35, both trained using data through period 28. I classified a transaction as illicit when its score was at least 0.50 and excluded transactions with unknown labels.

Evaluate periods	Model	N	Accuracy	Precision	Recall	F1	ROC-AUC
22–35	0929-1	11,705	0.9473	0.9502	0.6881	0.7982	0.9384
22–35	1019-6*	11,705	0.9471	0.9691	0.6723	0.7939	0.9641
29–35	0929-1	6,028	0.9827	0.9789	0.9256	0.9515	0.9971
29–35	1019-6*	6,028	0.9866	0.9749	0.9510	0.9628	0.9978

Table 5. Retrospective local validation at threshold 0.50

*Hybrid: uses five seeds with an 18-period rolling window and three-period fade.

Over periods 29–35, the hybrid caught 95.1% of illicit transactions, compared with 92.6% for 0929-1. Its alerts were slightly less precise, but its higher F1 indicated a better overall precision–recall balance.

Over periods 22–35, however, the hybrid produced more precise alerts while catching fewer illicit transactions, resulting in slightly lower F1.

These results show that the hybrid's stronger risk ranking did not consistently translate into better detection at the fixed cutoff. I therefore retained 0929-1 as an alternative.

3.3 Robustness and what did not transfer

What worked well & hypotheses

- *Recent-history training:* One explanation is that older patterns transfer less well, whereas an overly short window leaves too few illicit examples. A soft fade retains older observations at lower weight.
- *Unlabelled third class:* Labelling could be selective, so learning an observation-status class may exploit that selection structure and improve the licit–illicit boundary.
- *Bounded residual on a strong tree:* Trees capture nonlinear attribute relationships; graph messages add neighbourhood context. Each class-logit correction is bounded within ±1, limiting the neural adjustment. The binary illicit-versus-licit log-odds correction can therefore be as large as ±2.

What failed & hypotheses

- *Richer graph networks*: Attention, max/std aggregation and second-hop variants underperformed the simple hybrid publicly. A plausible explanation is that extra flexibility fitted neighbourhood patterns that did not transfer to later periods.
- *Further drift pruning*: Additional removal of drifting features scored 0.95345, below 1019-6. This differs from the initial ten-feature pruning retained in both recipes: later deletion may have removed useful signal along with instability.
- *Other unsuccessful directions*: Random Forest/Extra Trees teachers, pseudo-labels and covariate-shift reweighting underperformed in local experiments. Possible causes include insufficient adaptation, noisy self-generated targets and unstable weights.
- *Validation mismatch:* Small local improvements did not reliably transfer publicly. Historical windows overlap and precede the test distribution, so they cannot fully measure later changes. Reused windows and repeated submissions also make small gains vulnerable to selection bias.

Limitations

1. The 63 public submissions and reused temporal windows may create selection bias.
2. Nearby variants are not repeated draws from the same model, so their score spread cannot establish a noise interval or subtract a precise "lucky" component from 1019-6.
3. Low scores in later test periods do not reveal recall or missed fraud because test labels are hidden. Batch graph access also differs from real-time deployment, where later neighbours may be unavailable.

4. Conclusion and Reflection

This project changed how I think about fraud detection. At first, I focused mainly on improving ROC-AUC, but seeing small local gains fail to transfer to the leaderboard made me question what a better score actually tells me. Working with incomplete labels and changing transaction patterns also taught me to separate measured evidence from my own explanations.

More broadly, I came to understand that AI-driven fraud detection depends on the data available, whose activities receive labels, and how well past patterns represent future behaviour. A model may learn patterns in the labelling process as well as patterns associated with illicit activity. I also realised that prediction errors have different practical consequences: missed fraud can allow harmful activity to continue, while false alerts consume investigators' time and may subject legitimate users to unnecessary scrutiny. I now see effective fraud detection as an ongoing process of monitoring, reassessment and human judgement, with AI helping investigators decide where to further look into.

AI-use disclosure

I used Claude Code(Opus 5.5) and Codex(GPT-6.1 Sol) to assist with experimental analysis, code debugging, figure generation, and drafting this report. I reviewed the AI-generated outputs and take responsibility for the final code, analysis and report.